

Conceptos Técnicos

Guía de referencia para dirigir desarrollo de software musical

Pad Works · 2026 · Uso interno

1. FUNDAMENTOS — LO QUE VES VS LO QUE PROCESA

Frontend

→ Como la superficie de los pads de la MPC — lo que tocas y ves

Todo lo que el usuario ve e interactúa: botones, colores, animaciones, formularios. En Pad Works XPN es el HTML/CSS/JS que corre en el navegador. En una app móvil sería la pantalla.

Por qué importa: Cuando dices 'el botón no funciona' o 'se ve mal' estás hablando de frontend. Los prompts a Claude Code deben mencionar el elemento visual exacto y su comportamiento esperado.

Backend

→ Como el procesador interno de la MPC — invisible pero hace el trabajo pesado

El servidor, base de datos y lógica que corre 'detrás'. Procesa pagos, guarda usuarios, envía emails. En Pad Works: Supabase (base de datos) + Resend (emails) + Stripe (pagos) son el backend.

Por qué importa: Cuando algo falla en el pago o en el registro de usuarios, es un problema de backend. Requiere acceso al servidor, no solo al HTML.

API

→ Como el cable MIDI — protocolo estándar que conecta dos sistemas distintos

Interfaz que permite que dos programas se comuniquen. Stripe tiene una API que Pad Works usa para cobrar. Gumroad tiene una API. Claude tiene una API. Son 'enchufes' estandarizados.

Por qué importa: Cuando quieras conectar Pad Works con otro servicio (Spotify, MPC hardware, etc) necesitas una API. Si el servicio no tiene API pública, no se puede conectar automáticamente.

Plugin / VST

→ Como una expansión de hardware — se instala dentro de otro sistema y lo amplía

Software que se instala dentro de una DAW (Ableton, Logic, Pro Tools) y agrega funcionalidad. Los VST se programan en C++ con el framework JUCE. Son mucho más complejos que apps web — requieren optimización de audio en tiempo real.

Por qué importa: Hacer un plugin es 10x más complejo que Pad Works XPN. Requiere C++, JUCE, certificación de Apple/Windows, y testing exhaustivo con latencia de audio. Es Etapa 4 del roadmap.

DAW

→ Como una MPC pero completamente en software — el estudio completo

Digital Audio Workstation. Ableton, Logic, Pro Tools, FL Studio. Maneja audio en tiempo real, MIDI, efectos, mezcla. Proyectos de años de desarrollo con equipos de 20-50 personas y decenas de millones de dólares.

Por qué importa: Un DAW propio es una visión a 5-10 años. El camino correcto es construir herramientas que se integren con DAWs existentes primero — plugins, exportadores, importadores.

2. CONCEPTOS DE CÓDIGO — CÓMO DIRIGIR MEJOR

HTML / CSS / JS

→ HTML = estructura del kit · CSS = colores y diseño · JS = comportamiento

HTML define qué elementos existen (botón, panel, grid). CSS define cómo se ven (color, tamaño, posición). JavaScript define qué hacen (al hacer click, mostrar panel). Pad Works XPN es 100% estos tres lenguajes.

Por qué importa: Cuando un botón no funciona, el problema es en JS. Cuando algo se ve mal, es CSS. Cuando falta un elemento, es HTML. Ser específico en los prompts ahorra muchos tokens.

Función / Event listener

→ Como un pad programado — espera un trigger y ejecuta una acción

Una función es un bloque de código que hace algo. Un event listener 'escucha' si el usuario hace click, arrastra, presiona una tecla. Si el botón no responde, el event listener está desconectado o la función tiene un error.

Por qué importa: En prompts a Claude Code: 'el onclick del botón BEAT no llama a showModePanel()' es 10x más útil que 'el modo no funciona'.

Token

→ Como créditos de estudio — cada palabra procesada cuesta un poco

Claude cobra por tokens — unidades de texto procesadas. Un token es aproximadamente 4 caracteres. Leer un archivo de 5000 líneas cuesta muchos tokens. Prompts vagos obligan a Claude a leer más para entender.

Por qué importa: Prompts específicos = menos tokens. Mencionar el nombre exacto de la función, el número de línea si lo sabes, y el comportamiento esperado vs el actual. Evitar 'arregla todo' o 'no funciona'.

3. CÓMO DIRIGIR CLAUDE CODE — PLANTILLAS

■ Prompt vago (muchos tokens)

"El modo no funciona"

"Arregla los colores"

"Agrega una función nueva"

"El export no funciona bien"

■ Prompt eficiente (pocos tokens)

"El onclick del botón BEAT no muestra el panel .mode-preview. La función showModePanel() existe pero no se llama."

"El pad en posición A1 muestra color #333 pero debería mostrar el color de categoría kick (#8B4513) al 65% opacidad."

"Agrega un botón Cancel al panel #smart-suggest. Al hacer click llama a dismissSmartSuggest() que ya existe."

"El archivo XPM exportado no incluye el tag . La función buildXPM() en línea ~2400 debe agregar ese campo."